

In-memory Ledger: Architecture and Trade-offs

Ankit Kumar · companion to the ledger repository (see DECISIONS.md for the full decision log)

The ledger is a bitemporal, append-only, double-entry store. Every journal carries a value day (which day's balance it belongs to) and a booked day plus global sequence (when the system learned of it). Holds live in a separate append-only authorization log and never touch the money ledger. A single synchronized writer validates each event completely before appending, so every event is all-or-nothing. Current ledger balance and active holds are a write-through cache updated inside the append; value-dated daily closings are folded from the log because back-valued entries rewrite them; a reconciliation pass proves cache equals fold and that every currency nets to zero.

1. Append-only at scale

Where the design accumulates unbounded state

State	Grows with	How it is read today
Journal list	every posting, forever	<code>closingBalance</code> scans the whole list, all accounts, on every call
Fee look-back	age of the ledger	each end of day re-checks every day from Day 1 to today
Authorization log	every lifecycle record	latest state rebuilt by scanning all records on every authorization and settlement
Seen event ids	every event ever received	hash set, never expires
Rejections	every refusal	scanned linearly to de-duplicate end-of-day errors

What breaks first at 100x volume

The end-of-day fee run, not memory. It calls `closingBalance` once per account per elapsed day, and each call scans the entire, un-partitioned journal list. Cost per end of day is roughly $accounts \times days\ elapsed \times total\ journals$. With 100x accounts and 100x journals the work per close grows about 10,000x, and it keeps growing every day because the look-back starts at Day 1. The batch overruns its window long before memory is exhausted: fees and interest post late, and the next day's authorizations run against a ledger whose end-of-day state is incomplete. The single writer is not the first bottleneck: an in-memory append is microseconds, and the authorization path already reads $O(1)$ cached balances.

Cheapest structural change that defers it

A per-account, per-value-day net-movement index, maintained write-through. On append, add each posting's amount to `bucket[account][valueDay]` inside the same atomic step that already updates the balance cache. A closing balance becomes a prefix sum over at most $days$ buckets instead of a scan over all journals; a back-valued entry updates exactly one bucket, so restatement stays correct by construction. The fee run drops from $A \times D \times J$ to $A \times D$, and `reconcile()` extends naturally to "index equals fold".

This defers the problem rather than removing it: D still grows. The next step is a **back-valuation horizon** (see §2): once no entry may be value-dated before day H , everything up to H is sealed into one carried-forward balance per account, the look-back becomes bounded, and sealed journals move to cold storage. That is archiving, not deletion; the log stays append-only. Seen event ids get the same treatment: retained for a window at least as long as the longest upstream retry, then expired.

2. Value-dated entries in production

A value-dated entry changes a balance that other processes have already consumed. In a UAE-licensed bank, each of those consumers is an operational or regulatory exposure.

Surface	What a back-valued entry does to it
Fees and interest	Retroactively creates or removes fee triggers and changes interest accrued on past days. This build shows both: fees appear for days that had already closed, and interest is computed on restated balances.
Customer statements and conduct	Balances on statements already sent no longer match the account. Fees triggered, or left standing, by an entry later reversed are a fair-treatment and fee-transparency issue under the Central Bank of the UAE's consumer protection framework, and a likely complaint.

Decisions already taken	Authorization approvals and declines were made on the balance known at the time and cannot be re-run. A customer can be declined because of a debit that is later reversed.
Accounting period close	An entry dated into a closed month changes figures already closed in the general ledger and the sub-ledger to GL reconciliation, and potentially financial reporting.
Regulatory returns	Returns to the Central Bank are struck on balances at reporting dates; back-valuing into a reported period means explaining or resubmitting differences.
AML/CFT	Back-dating moves the apparent timeline of funds. Transaction monitoring must evaluate on both value time and booking time, and both, plus who approved the back-dating, must be retained for the statutory record-keeping period under the UAE AML law (Federal Decree-Law No. 20 of 2018).
Calendar	Value dates must respect the UAE Saturday–Sunday weekend, public holidays, cut-off times, scheme settlement dates and FX value dates for non-AED currencies. The model has integer days only.
Internal fraud	Back-valuation is a classic manipulation channel: dating an entry so an account looks funded on a reporting date, earns interest, or avoids a fee.

Control to add before go-live: a back-valuation approval gate. Any entry whose value date is more than one business day before its booking date, or falls in a closed statement or accounting period, is held as pending instead of posted. Posting requires maker-checker approval with a reason code. On approval the system automatically computes and records the customer impact (fees triggered or removed, interest difference) and routes any refund decision; every such entry appears on a daily exception report reviewed by operations and compliance. The same threshold is the horizon that makes §1's sealing possible.

3. Authorization lifecycle

In this model an authorization has three states and only two outcomes that end it: **DECLINED** at request and **SETTLED**. Every other way a real authorization ends leaves an **APPROVED** hold active forever, reducing available balance indefinitely. That is the most important gap to close.

Ending	What the model does today	Real-world scenario	Behaviour I would mandate
Declined	DECLINED record with the balances it was decided on; never holds funds	Insufficient funds; also a decline caused by a debit later reversed	Final for that request. If a back-valued correction later shows the decline was caused by bank error, flag it in the customer-impact review; never retro-approve.
Settled for less than the hold	Settles, releases the whole hold	Final amount lower: fuel, hotel, restaurant	Release the remainder immediately, unless the authorization is flagged for multi-capture (split shipment), in which case keep it until the final capture or expiry.
Never settled	Stays APPROVED forever	Merchant never captures: cancelled order, abandoned pre-authorization, deposit	Expiry by merchant category (shorter for retail, longer for hotel and car rental, per scheme rules): new EXPIRED record, hold released. A later presentment is accepted and flagged as late, not rejected.
Merchant reversal	Not modelled; hold stays	Customer cancels; terminal timeout reversal	An authorization-reversal event (full or partial) that releases the hold, idempotent on the network message id.
Settlement above the hold	Settlement refused; hold stays active	Tips, incremental charges	Tolerance by merchant category. Above it, post the presentment (the issuer is already liable to the scheme), release the hold and raise a dispute case, instead of leaving both a refused settlement and a stale hold.
Settlement refused for another reason	Wrong account, already settled, or declined: refused; any hold untouched	Duplicate presentment; mis-routed clearing record	Exact duplicates rejected idempotently; everything else goes to a suspense account and exception queue with an SLA, never silently dropped.

Card or account blocked	Not modelled	Lost card, account closure	Block new authorizations; existing holds remain until settled or expired; settlements against them are still honoured.
-------------------------	--------------	----------------------------	--

The mirror case, a settlement with no authorization at all, is refused in this build. In production it is a legitimate force-post or late presentment and belongs in the same suspense-and-review path.

4. What I cut and why

Simplification	Why it was cut	Production risk it defers
No persistence or durability	Out of scope by brief	A crash loses the ledger. Needs a durable event store with write-ahead logging; the cache must be rebuilt from it on start.
One global synchronized writer	Simplest correct answer to the check-then-hold race	Throughput ceiling. Production: one writer per account partition; multi-account transfers then need a coordination protocol.
Reconciliation in the same process	Proves the cache logic	Cannot detect storage corruption. Production reconciles independent stores (ledger vs GL vs scheme files).
Integer days, no timestamps or calendar	The window is six abstract days	Wrong value dates around weekends, holidays and cut-offs; no time zone handling.
No FX	No rates in scope	The AED fee cannot be booked on a BHD account; it is recorded as an error instead.
No hold expiry, authorization reversal, incremental or multi-capture	Not in the event stream	Holds leak forever (§3).
No fee waiver or refund on reversal	No rule authorises it; it is policy	Customers pay fees caused by entries later reversed: conduct and complaint risk (the deliberately failing test).
Unbounded back-valuation, no period close	Six-day window	Unbounded look-back cost (§1) and uncontrolled restatement (§2).
Unlimited overdraft on posted debits	Only authorizations are gated by the rules	No overdraft limit, no credit risk control on posted debits.
Simple interest model	Daily rate given directly	No day-count convention, tiering, tax or accrual booking; accruals exist only at capitalisation.
Reversal of credits and debits only, whole journal	Enough for the stream	No partial reversal, settlement reversal or chargeback flow.
In-memory idempotency set; rejected ids are burned	Cheap protection against double posting	Grows forever; a corrected message must carry a new id.
No access control, maker-checker or tamper evidence	No users in scope	Anyone with code access can append; no hash chain to prove history was not altered outside the API.
Global constants, no product configuration	One product in scope	Fee, rate and tolerance cannot vary by product or customer segment.